

## Mark2

A powerful and flexible Markdown converter that transforms Markdown files into multiple output formats including HTML, EPUB, PDF, and ConTeXt (TeX).

## Features

- **Multiple Output Formats:** Convert Markdown to HTML, EPUB, PDF, ConTeXt (TeX), or Markdown
- **MyST Markdown Support:** Extended Markdown syntax with MyST (Markedly Structured Text) features
- **Front Matter:** Support for YAML front matter metadata
- **Footnotes:** Automatic footnote handling and rendering
- **Custom Heading IDs:** Automatic generation of heading identifiers for cross-referencing
- **EPUB Customization:** Add custom covers and stylesheets to EPUB output
- **Flexible I/O:** Read from files or stdin, write to files or stdout

## Plugins

Mark2 includes several powerful plugins that extend standard Markdown capabilities:

- [YAML Front Matter](#) - Document metadata using YAML
- [Footnotes](#) - Reference-style and inline footnotes
- [MyST Roles](#) - Inline semantic extensions
- [MyST Directives](#) - Block-level extensions (comments, breaks, targets)
- [Container Blocks](#) - Custom block containers for admonitions and notes
- [Superscript and Subscript](#) - Scientific notation support
- [Strikethrough](#) - Mark text as deleted or struck through
- [Page Breaks](#) - Control pagination in PDF and EPUB output
- [Custom Heading IDs](#) - Automatic and custom heading identifiers
- [Tables](#) - GitHub-Flavored Markdown tables
- [Block Attributes](#) - Add HTML attributes to block elements

See the [Extensions Over CommonMark](#) section for detailed documentation and examples.

## Installation

```
# Clone the repository
git clone https://github.com/geckoblu/mark2.git
cd mark2
```

```
# Install dependencies (requires Python 3.x)
pip3 install markdown-it-py mdit-py-plugins
```

## Usage

### Basic Usage

```
# Convert Markdown to HTML (default)
./mark2.sh input.md
```

```
# Specify output format
./mark2.sh input.md -f epub
./mark2.sh input.md -f pdf
./mark2.sh input.md -f tex
```

```
# Specify output file
./mark2.sh input.md -o output.html
```

```
# Read from stdin
cat input.md | ./mark2.sh - -f html
```

### Command-Line Options

positional arguments:

INPUT\_FILENAME           input Markdown (.md) file to  
convert (use '-' for stdin)

options:

-h, --help               show this help message and  
exit  
-f {html,epub,pdf,tex}, --format {html,epub,pdf,tex}  
                          output format (choices: html,  
epub, pdf, tex) [default: html]  
-o OUTPUT\_FILENAME, --output-filename OUTPUT\_FILENAME  
                          output file name (use '-' for  
stdout) [default: input name with format extension]  
-q, --quiet               suppress non-error messages  
--link-check              extract and check all links in  
the document

EPUB options:

--epub-cover EPUB\_COVER  
                          cover image for epub  
--epub-generatecover     generate cover for epub  
--epub-stylesheet EPUB\_STYLESHEET  
                          stylesheet for epub  
--epub-split-at-header {h1,h2,h3,h4,h5,h6}  
                          header level at which to split  
content into separate pages [default: h2]

PDF options:

```

--pdf-keep-tex          keep intermediate .tex file
generated during PDF output
--pdf-page-format {A4,A5}
                        page format for PDF output
(choices: A4, A5) [default: A4]
--pdf-font-size PDF_FONT_SIZE
                        main body font size for PDF
output, e.g. '12pt' [default: format-dependent]
--pdf-font-name PDF_FONT_NAME
                        main body font name for PDF
output, e.g. 'libertinus' [default: format-dependent]

```

## Examples

```

# Generate HTML with automatic output filename
./mark2.sh document.md

# Create an EPUB with custom cover and stylesheet
./mark2.sh book.md -f epub --epub-cover cover.jpg
--epub-stylesheet style.css

# Convert to PDF
./mark2.sh report.md -f pdf

# Convert to PDF with a custom page format, font, and
font size
./mark2.sh report.md -f pdf --pdf-page-format A5 --pdf-font-name
libertinus --pdf-font-size 11pt

# Generate ConTeXt source
./mark2.sh article.md -f tex -o article.tex

# Process stdin to stdout
cat notes.md | ./mark2.sh - -f html > notes.html

```

## Supported Markdown Features

Mark2 uses the `markdown-it-py` parser with support for:

- **Standard Markdown:** Headers, emphasis, lists, links, images, code blocks
- **Tables:** GFM-style tables
- **Footnotes:** Reference-style footnotes
- **Front Matter:** YAML metadata blocks
- **Heading IDs:** Automatic or custom heading identifiers
- **MyST Extensions:** Extended syntax for scientific and technical writing

## Extensions Over CommonMark

Mark2 extends the [CommonMark specification](#) with several powerful features for scientific, technical, and professional documentation. Below is a comprehensive guide to these extensions.

### 1. YAML Front Matter

Add metadata to your documents using YAML front matter at the beginning of your file.

#### Syntax:

```
---
title: Document Title
author: John Doe
date: 2024-01-22
id: ISBN<9788831550420>
publisher: Acme
subject: Test,Emphasis
description: |
  This is a multi-line
  description that preserves
  line breaks.
summary: >
  This is a long paragraph
  that will be folded into
  a single line with spaces.
---
```

# Content starts here

#### Features:

- Must be the first thing in the file
- Enclosed by --- markers
- Supports standard YAML syntax including multi-line values
- Metadata can be used by renderers (especially EPUB and PDF)

#### PDF-specific keys:

The ConTeXt/PDF renderer also reads the following optional keys, which override the page-format defaults (A4/A5) but are overridden by the matching --pdf-\* CLI option (priority: CLI > front matter > format default):

```
---
pdf-font-size: 11pt
pdf-font-name: libertine
pdf-header-at-recto: h2,h3
---
```

- `pdf-font-size / pdf-font-name`: main body font size/name
- `pdf-header-at-recto`: comma-separated heading levels (h1, h2, h3, h4) that must always start on a right-hand (recto) page
- Each key also accepts a page-format-specific variant that takes precedence over the generic one, e.g. `pdf-a4-font-size`, `pdf-a5-font-name`, `pdf-a4-header-at-recto`

## 2. Footnotes

Create reference-style footnotes with automatic numbering and positioning.

### Syntax:

Text with a footnote<sup>[^1]</sup> and another<sup>[^note]</sup>.

[^1]: This is the first footnote.

[^note]: This is a named footnote.

And finally<sup>[This is an inline note]</sup>

### Inline Footnotes:

Text with an inline footnote<sup>[This note appears inline]</sup>.

### Features:

- Automatic numbering
- Support for both numeric and named references
- Footnote definitions can appear anywhere in the document
- Inline footnotes using `^[ . . . ]` syntax
- Footnotes are collected and rendered at the end of the document

### Complete Example:

The Manhattan Project<sup>[^manhattan]</sup> was a research project during World War II.

Quantum mechanics<sup>[A fundamental theory in physics]</sup> revolutionized science.

Einstein's famous equation<sup>[^einstein]</sup> changed our understanding of energy.

[^manhattan]: The project ran from 1942 to 1946 and produced the first nuclear weapons.

[^einstein]:  $E = mc^2$  relates mass and energy, published in 1905.

### Rendered footnotes appear at the end:

1. The project ran from 1942 to 1946 and produced the first nuclear weapons.
2. A fundamental theory in physics
3.  $E = mc^2$  relates mass and energy, published in 1905.

### 3. MyST Roles (Inline Extensions)

Apply semantic meaning or special formatting to inline text using MyST roles.

#### Syntax:

`{role-name}`content``

#### Features:

- Role names can contain letters, numbers, underscores, hyphens, plus signs, and colons
- Support for multiple backticks: `{role}content with backtick`
- Newlines in content are converted to spaces
- Can be escaped with backslash: `\{role}content``

#### Special Roles:

Some roles don't require content or backticks:

`{line-break}` or `{br}`

- `{line-break}` or `{br}` - Insert a line break without content
- These roles are used without backticks or content
- `{br}` is an alias for `{line-break}`
- can be used to insert line break in headers

#### Complete Example:

This is `{emphasis}`very important`` information.  
The file is located at `{code}`/usr/local/bin``.  
Press `{kbd}`Ctrl+C`` to copy.  
Chemical formula: `{sub}`H2O`` or using built-in: `H~2~O`  
Mathematical expression: `{math}`x^2 + y^2 = z^2``

# Long Title`{br}`With Line Break

First line of text`{line-break}`Second line after break.

### 4. MyST Directives (Block Extensions)

#### 4.1 Line Comments

Add comments to your Markdown that won't appear in the output.

#### Syntax:

```
% This is a comment
% Comments can span multiple lines
% when each line starts with %
```

Regular content here.

#### Features:

- Lines starting with % are treated as comments
- Multiple consecutive comment lines are combined into one HTML comment block
- Leading whitespace is stripped
- Does not work inside code blocks

**Example:**

```
% TODO: Add more details about the methodology
% Author: Review this section before publication

# Introduction

% This section needs peer review
The research methodology follows established practices.
```

## 4.2 Block Breaks

Create visual separators or section breaks.

**Syntax:**

Content above

```
+++
```

Content below

**Features:**

- Minimum 3 plus signs: +++
- Can use more plus signs: ++++++++
- Rendered as `<hr class="myst-block">`
- Useful for visual or semantic section breaks in longer documents
- Unlike ---, does not create a page break in PDF or EPUB output

**Example:**

```
## Section 1

First part of the content.

+++

## Section 2

Second part after a visual break.

+++++++

## Section 3
```

Another section with a different break style.

### 4.3 Targets (Labels)

Create named anchors for cross-referencing.

**Syntax:**

```
(my-label)=  
# Section Title
```

Reference the label elsewhere: {ref}`my-label`

**Features:**

- Labels must be on their own line
- Typically placed before headings
- The default renderer outputs a labeled anchor element
- {ref} is rendered as a styled inline role; it does not automatically resolve the label to a link
- Use a normal Markdown link, such as [Introduction](#introduction), for an explicit internal link

**Complete Example:**

```
(introduction)=  
# Introduction
```

This is the introduction section.

```
(methodology)=  
## Research Methodology
```

As discussed in the [Introduction](#introduction), our approach...

```
(results)=  
## Results
```

The results are analyzed in detail. See [Research Methodology](#methodology) for context.

## 5. Container Blocks

Create custom block-level containers with semantic meaning.

**Syntax:**

```
::: container-name  
Content with **markdown** support.  
:::
```

**Features:**

- Minimum 3 colons: :::
- Content inside supports full Markdown syntax
- Containers can be nested with more colons: ::::, :::::, etc.
- Useful for admonitions, warnings, notes, etc.

### Complete Examples:

```
::: note
This is an important note with emphasis.
:::
```

```
::: warning
Warning: Proceed with caution!
:::
```

```
::: tip
Tip: Use keyboard shortcuts to save time.
:::
```

```
:::: info
Nested example:
::: example
Code example goes here
:::
::::
```

### Rendered as:

```
<div class="note">
<p>This is an important note with
<em>emphasis</em>.</p>
</div>
```

## 6. Superscript and Subscript

### Superscript Syntax:

```
E = mc^2^
This is a footnote reference^1^
```

### Subscript Syntax:

```
H~2~O
Temperature at T~0~
```

### Complete Examples:

```
% Scientific formulas
Water molecule: H~2~O
Einstein's equation: E = mc^2^
Glucose: C~6~H~12~O~6~
```

% Mathematical expressions  
 $x^2 + y^2 = z^2$   
 $\text{Log}_{10}(100) = 2$

% Chemical reactions  
 $2\text{H}_2 + \text{O}_2 \rightarrow 2\text{H}_2\text{O}$

% Ordinal numbers  
1<sup>st</sup>, 2<sup>nd</sup>, 3<sup>rd</sup>, 4<sup>th</sup>

## 7. Strikethrough

Mark text as deleted or struck through.

### Syntax:

`~~strikethrough text~~`

### Examples:

This is `~~deleted~~` text.  
The price was `~~$100~~` \$80.

% Showing corrections  
`~~Incorrect information~~` Corrected information

% Task lists  
- `~~Completed task~~`  
- Pending task  
- `~~Another completed task~~`

% Revisions  
The meeting is scheduled for `~~Monday~~` Tuesday at 3 PM.

### Rendered as:

This is `<s>deleted</s>` text.  
The price was `<s>$100</s>` \$80.

## 8. Page Breaks

Insert page breaks for PDF and print output.

### Syntax:

Content on first page

---

Content on second page

### Features:

- Three consecutive dashes: ---
- Only horizontal rules with dashes become page breaks
- Thematic breaks with asterisks (\*\*\*) remain unchanged
- Rendered differently depending on output format (special handling for PDF/EPUB)

### **Complete Example:**

```
# Chapter 1: Introduction
```

Content of chapter one goes here.  
Multiple paragraphs can be included.

```
---
```

```
# Chapter 2: Background
```

This chapter starts on a new page in PDF/EPUB output.

```
---
```

```
# Chapter 3: Methodology
```

Another page break before this chapter.

```
***
```

Note: Three asterisks create a thematic break, not a page break.

## 9. Custom Heading IDs

Automatically generates or allows custom IDs for headings.

### **Automatic IDs:**

```
# My Section Title
```

By default, Mark2 generates sequential IDs such as `<h1 id="toc_id_1">My Section Title</h1>`.

### **Custom IDs (using targets):**

```
(custom-id)=
```

```
# My Section Title
```

This creates a separate target before the heading: `<div class="myst-target"><a href="#custom-id">(custom-id)=</a></div>`. It does not replace the heading's generated ID.

### **Features:**

- Sequential automatic IDs for headings
- Separate named targets using (name) = syntax
- Enables internal linking and table of contents generation

### Complete Example:

```
# Introduction to Machine Learning
% Auto-generated ID: #toc_id_1

(custom-section)=
## What is Machine Learning?
% Separate target: #custom-section; the heading keeps
its generated ID

### Supervised Learning
% Auto-generated ID: #toc_id_3

(neural-nets)=
### Neural Networks
% Separate target: #neural-nets; the heading keeps its
generated ID

You can link to sections:
- [Introduction](#toc_id_1)
- [Machine Learning](#custom-section)
- [Neural Networks](#neural-nets)
```

## 10. Tables (GFM Style)

Standard GitHub-Flavored Markdown tables.

### Syntax:

| Header 1 | Header 2 | Header 3 |
|----------|----------|----------|
| Cell 1   | Cell 2   | Cell 3   |
| Cell 4   | Cell 5   | Cell 6   |

### Alignment:

| Left | Center | Right |
|------|--------|-------|
| L1   | C1     | R1    |
| L2   | C2     | R2    |

### Complete Examples:

```
% Simple table
```

| Name  | Age | City     |
|-------|-----|----------|
| Alice | 30  | New York |

|         |    |             |
|---------|----|-------------|
| Bob     | 25 | Los Angeles |
| Charlie | 35 | Chicago     |

% Table with formatting

| Feature               | Status | Priority |
|-----------------------|--------|----------|
| :-----                | :----- | :-----   |
| <b>**Bold text**</b>  |        | High     |
| <i>*Italic*</i>       |        | Medium   |
| <del>~~Strike~~</del> |        | Low      |

% Aligned table

| Product | Price  | Quantity |
|---------|--------|----------|
| :-----  | :----- | :-----   |
| Apple   | \$2.50 | 10       |
| Banana  | \$1.20 | 25       |
| Orange  | \$3.00 | 8        |

## 11. Block Attributes

Add HTML attributes (classes, IDs, and key-value pairs) to block elements.

### Syntax:

```
{#id .class1 .class2 key="value"}
Block content here
```

### Features:

- `.classname` - Specifies CSS classes (multiple classes can be added)
- `#identifier` - Specifies an ID (only one ID per element; last one wins if multiple)
- `key="value"` or `key=value` - Adds key-value attributes
  - Quotes optional for alphanumeric values with `_`, `:`, or `-`
  - Backslash escapes work inside quoted values
- `%comment%` - Add comments within attributes
- Attributes must be on their own line, before the target block
- Multiple attribute blocks stack (classes accumulate, later values override earlier ones)

### Examples:

#### Basic usage:

```
{.warning #main-warning}
```

This paragraph will have `class="warning"` and `id="main-warning"`

#### Multiple classes and ID:

```
{#section-intro .highlight .bordered}
# Introduction
```

This heading will have `id="section-intro"` and `class="highlight bordered"`

### Stacking attributes:

```
{.primary}  
{.large #header}  
# Welcome
```

Results in: id="header" class="primary large"

### Key-value attributes:

```
{style="color: red;" data-toggle="collapse"}  
This paragraph has custom attributes
```

### Rendered as:

```
<p style="color: red;" data-toggle="collapse">This  
paragraph has custom attributes</p>
```

### With comments:

```
{.note %styling% style="border: 1px solid blue;" %end%}  
Important information
```

### Advanced Examples:

```
% Adding data attributes for JavaScript  
{data-toggle="modal" data-target="#myModal"}  
Click here to open modal
```

```
% Styling specific paragraphs  
{.lead style="font-size: 1.25em;"}  
This is a lead paragraph with larger text.
```

```
% Multiple stacked attributes  
{.container}  
{.row}  
{#main-content .col-md-8}  
Main content area with Bootstrap classes
```

```
% Combining with other features  
{#important-table .table .table-striped}  
| Column 1 | Column 2 |  
| ----- | ----- |  
| Data 1   | Data 2   |
```

### Complete Example

Here's a comprehensive example using multiple extensions:

```
---  
title: Research Paper
```

```
author: Dr. Jane Smith
date: 2024-01-22
---
```

```
% This is a comment explaining the document structure
% It won't appear in the output
```

```
(introduction)=
# Introduction
```

```
This paper discusses the formula  $E = mc^2$  and the
importance
of  $H_2O$  in biological systems[1]. ~~Previous theories~~
have been superseded.
```

```
{emphasis}`Note:` This is a MyST role for emphasis.
```

```
{.alert .alert-warning}
::: warning
Please read this section carefully.
:::
```

```
## Methods
```

```
{#methods-section .highlighted}
The experimental setup is described below.
```

```
+++
```

```
[1]: Water is essential for all known forms of life.
```

```
---
```

```
# Conclusion
```

```
See the {ref}`introduction` for background information.
```

This example demonstrates:

- YAML front matter
- Comments
- Custom heading targets
- Superscript and subscript
- Strikethrough
- Footnotes
- MyST roles
- Container blocks
- Block attributes
- Block breaks

- Page breaks
- Internal references

## Architecture

The project is structured as follows:

- `src/mark2/`: Main package
  - `main.py`: Entry point and conversion orchestration
  - `args.py`: Command-line argument parsing
  - `plugins/`: Custom markdown-it plugins
    - ★ `footnote_plugin.py`: Enhanced footnote handling
    - ★ `headingsid_plugin.py`: Heading ID generation
    - ★ `yaml_parser.py`: YAML front matter parsing
  - `renderer/`: Output format renderers
    - ★ `html.py`: HTML renderer
    - ★ `md.py`: Markdown renderer
    - ★ `pdf.py`: PDF renderer
    - ★ `context.py`: ConTeXt renderer
    - ★ `epub/`: EPUB-specific rendering

## Development

### Running Tests

```
# Run tests
export PYTHONPATH=./src:./tests
pytest tests/
```

see the tests [README](#) for detailed examples

### Direct Python Usage

```
# Set PYTHONPATH and run directly
export PYTHONPATH=./src
python3 src/mark2/main.py input.md
```

## References

- [CommonMark](#)
- [CommonMark Spec](#)
- [Markdown-It-Py](#)
- [Markdown-It-Py Plugin Extensions](#)
- [markdown-it-py \(source\)](#) - Python port of markdown-it
- [markdown-it](#) - Original JavaScript implementation
- [MyST Markdown](#) - MyST quickstart guide (JavaScript)
- [MyST Specification \(myst-parser\)](#) - Typography and syntax

- [MyST Specification \(mystmd\)](#) - MyST typography guide (JavaScript)
- [ConTeXt Garden](#) - ConTeXt documentation

## License

This project is released under the MIT License, see [LICENSE](#) for details.

## Contributing

Contributions are welcome! Please feel free to submit issues or pull requests.

